Waiting and Blocking in a Linux Driver

In an era in which there is an ever increasing demand for speed and agility, waiting is the last thing anybody wants. Yet, waiting is an inevitable part of the system. Speed and waiting are two sides of the same coin. For example, though traffic signals make us wait, they are needed to ensure safety. So, why do we need to wait in the driver? What mechanisms does the kernel provide for waiting? Read on to find the crux of waiting in Linux drivers.

Many a time, while interacting with the hardware, a process needs to wait for some event. The event can be the specified period of time before which the device is available for use. To give an example, while interacting with an LCD, after sending one command, there is a need to wait for a specified length of time, before another command can be sent. The event can be the arrival of data from some device such as a disk. The event can very well be waiting for some shared resource to be available. All such scenarios require the process to wait, until the event occurs.

Process states in Linux

A process goes through various stages during its life cycle. At any point of time, a process can be in any one of the states mentioned below.

- TASK_RUNNING: The process is in the run queue or is running.
- TASK_INTERRUPTIBLE: The process is waiting for an event, but can be woken up by a signal.
- TASK_UNINTERRUPTIBLE: This is similar to TASK_INTERRUPTIBLE, but the receipt of the signal has no effect.
- TASK_ZOMBIE: The process is terminated, but not cleaned up yet.
- TASK_STOPPED: The process was stopped by the debugger.

For the process to be scheduled to run, it needs to be in the run queue. This in turn means that the process state should be TASK_RUNNING in order for the scheduler to run it. TASK_INTERRUPTIBLE and TASK_UNINTERRUPTIBLE correspond to the waiting process states, wherein the process moves out of the run queue and wouldn't be scheduled by the scheduler till it moves back to the run queue.

Linux kernel wait mechanisms

The basic wait mechanism which the kernel provides is the *schedule()* API, where the process voluntarily gives up the processor by invoking the scheduler. Let's look at the following programming example to understand the usage of this API:

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <asm/uaccess.h>
#include <linux/wait.h>
#include <linux/sched.h>
#include <linux/delay.h>
#define FIRST_MINOR 0
#define MINOR_CNT 1

static dev_t dev;
static struct cdev c_dev;
static struct class *cl;

int open(struct inode *inode, struct file *filp)
{
printk(KERN_INFO "Inside open \n");
    return 0;
}

int release(struct inode *inode, struct file *filp)
{
    printk (KERN_INFO "Inside close \n");
    return 0;
}

ssize_t read(struct file *filp, char *buff, size_t
count, loff_t *offp)
{
    printk(KERN_INFO "Inside read\n");
    printk(KERN_INFO "Scheduling out\n");
    schedule();
    printk(KERN_INFO "Woken up\n");
```